COLUMBIA ENGINEERING

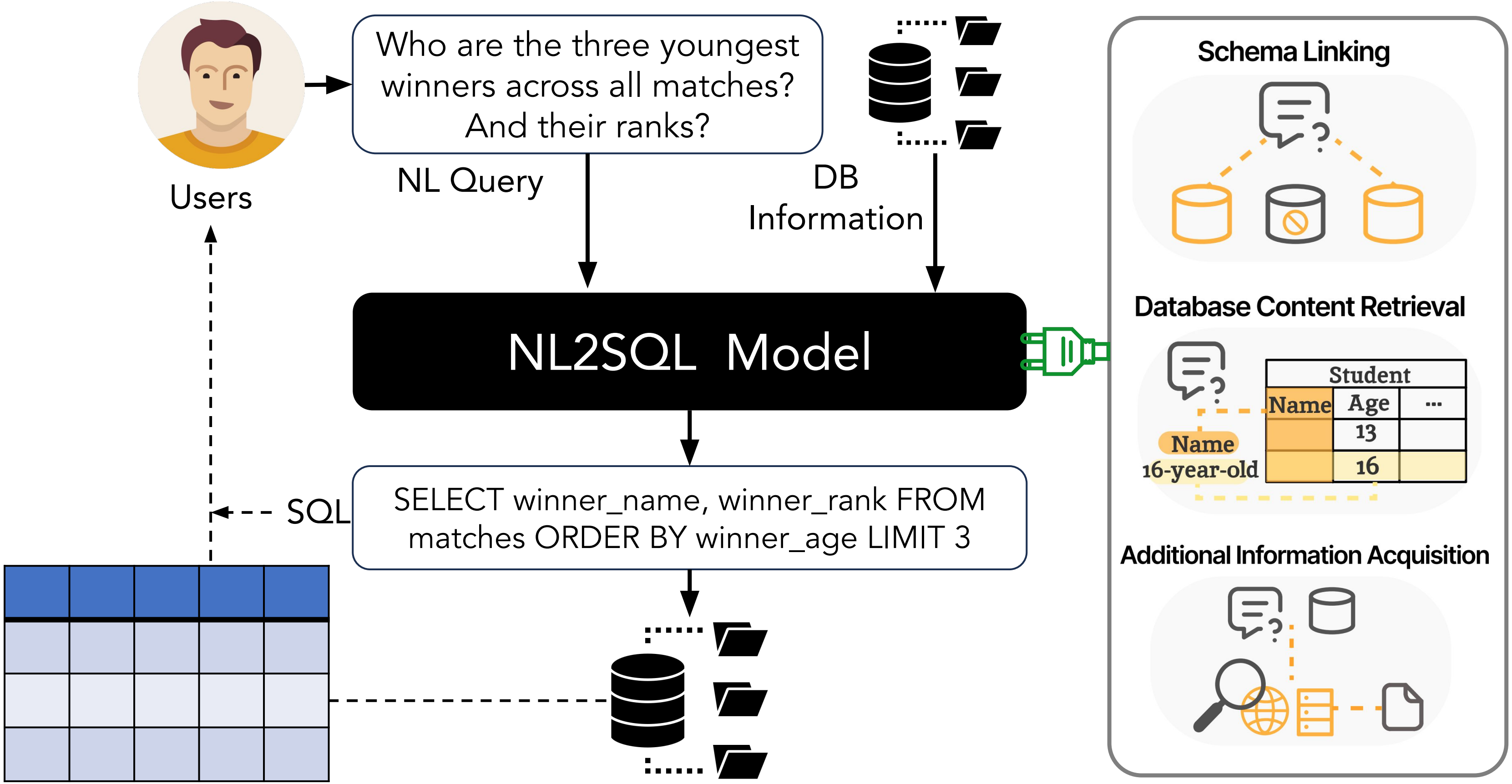
Natural Language to SQL(NL2SQL) in the LLM Era

State of the Art and Open Problems

Jinhui (Jensen) Liu

Adapted from slides by Yuyu Luo

NL2SQL (Text-to-SQL): Bridges Humans and Databases

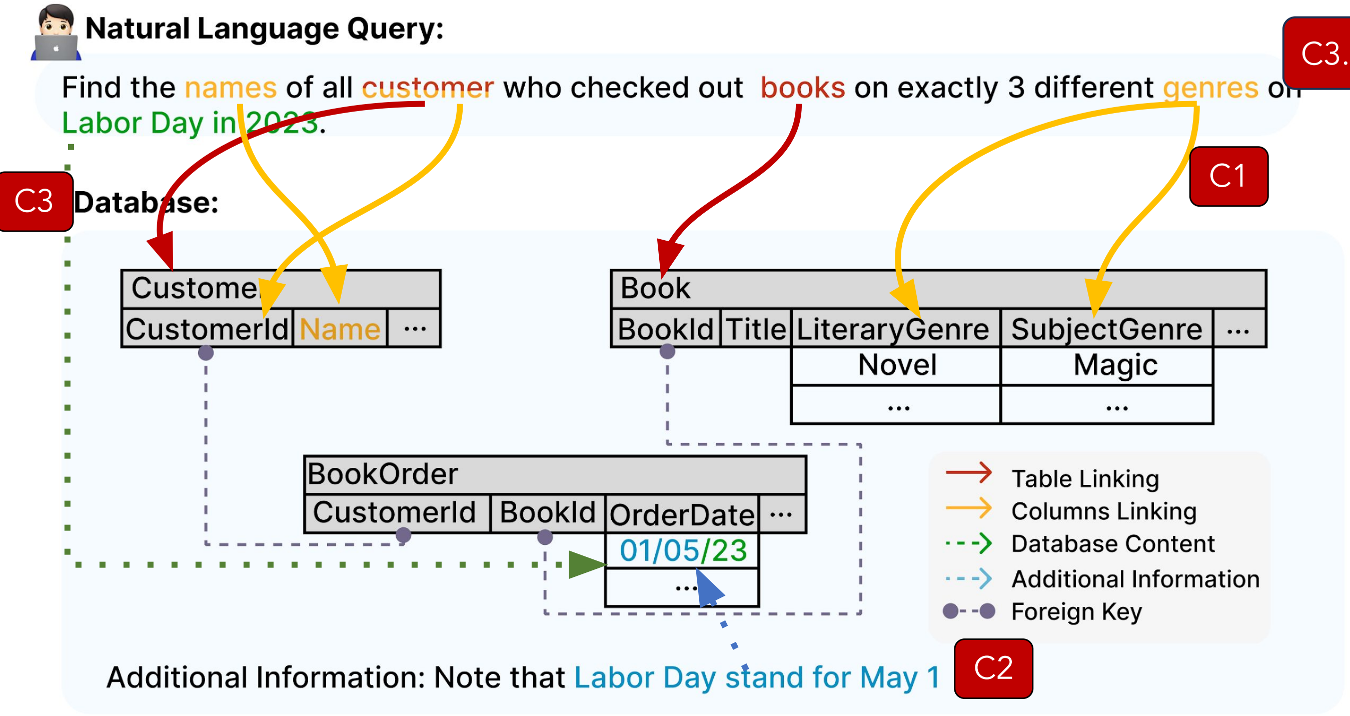


Task Challenges

C1. Ambiguous NL Query

C2. Requiring Domain Knowledge

C3. Complex Database Schema

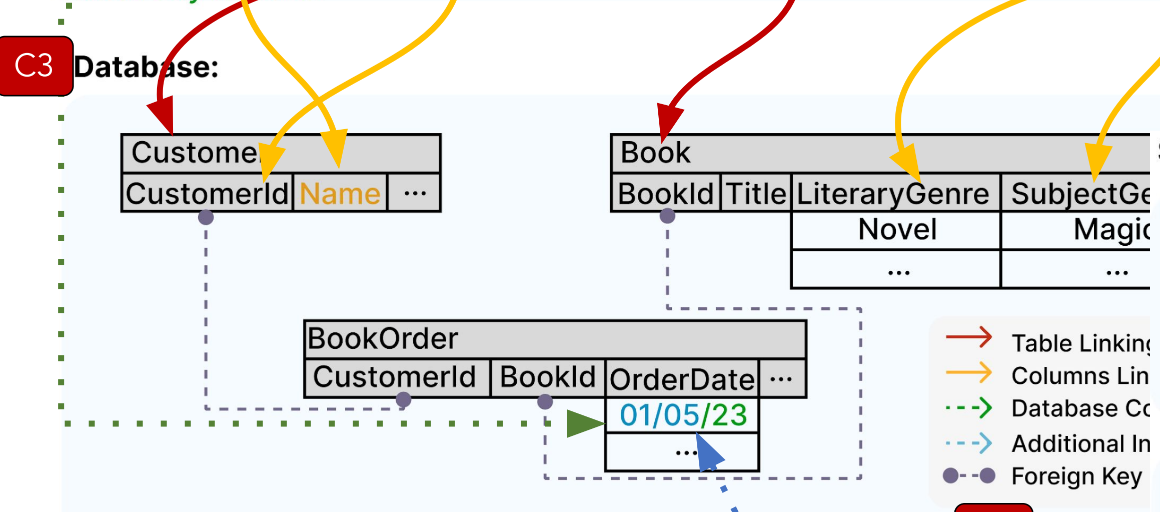


Task Challenges

- C1. Ambiguous NL Query
- C2. Requiring Domain Knowledge
- C3. Complex Database Schema
- C4. Multiple Possible SQL Queries

Natural Language Query:

Find the names of all customer who checked out books on exactly 3 different genres on Labor Day in 2023.



Additional Information: Note that Labor Day stand for May 1

C2

SQL:

```
SELECT Name
FROM Customer
NATURAL JOIN BookOrder NATURAL JOIN Book
WHERE OrderDate = '01/05/23'
GROUP BY CustomerId, Name
HAVING COUNT(DISTINCT SubjectGenre) = 3;
```

```
SELECT Name
FROM Customer
WHERE CustomerId = (SELECT CustomerId
FROM BookOrder NATURAL JOIN Book
WHERE OrderDate = '01/05/23'
GROUP BY CustomerId
HAVING COUNT(DISTINCT SubjectGenre)=3);
```

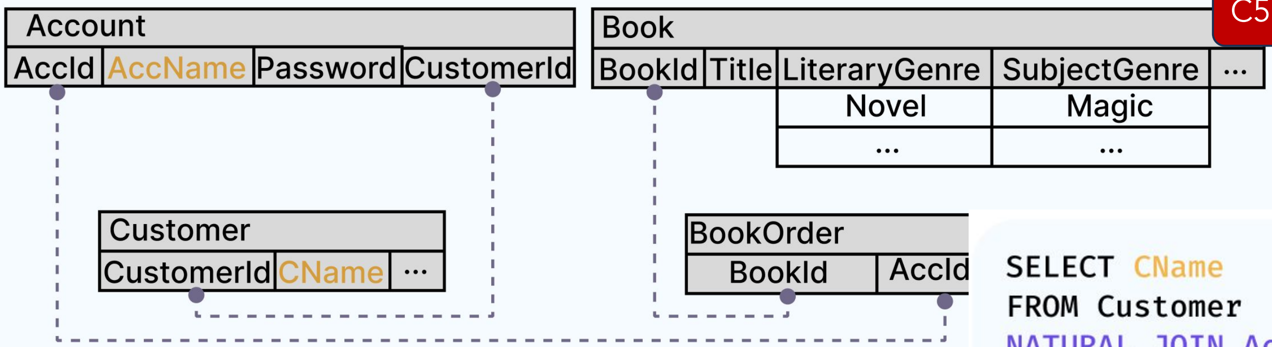
Task Challenges

- C1. Ambiguous NL Query
- C2. Requiring Domain Knowledge
- C3. Complex Database Schema
- C4. Multiple Possible SQL Queries
- C5. Database Schema Dependency

Natural Language Query:

Find the names of all customer who checked out books on exactly 3 different genres on Labor Day in 2023.

Database:



Additional Information: Note that Labor Day stand for May 1

```sql
SELECT CName
FROM Customer
NATURAL JOIN Account
NATURAL JOIN BookOrder
NATURAL JOIN Book
WHERE OrderDate = 'May 1st 2023'
GROUP BY CustomerId, CName
HAVING COUNT(DISTINCT SubjectGenre) = 3;
```

Challenges

NL2SQL Challenges

- Uncertain NL Query
  - *Lexical Ambiguity*
  - *Syntactic Ambiguity*
  - *Under-specification*
  - *User Mistakes*
- Complex Database and Dirty Content
  - *Complex Relationships Among Tables*
  - *Ambiguity in Attributes and Values*
  - *Domain-Specific Schema Designs*
  - *Large and Dirty Database Values*
- NL2SQL Translation
  - *Free-form NL vs. Constrained and Formal SQL*
  - *Multiple Possible SQL Queries*
  - *Database Schema Dependency*
- ***Technical Challenges*** in Developing NL2SQL Solutions
  - *Model Efficiency*
  - *SQL Efficiency*
  - *Insufficient and Noisy Training Data*
  - *Cost-effective Solution*
  - *Data Privacy*
  - *Trustworthiness and Reliability*

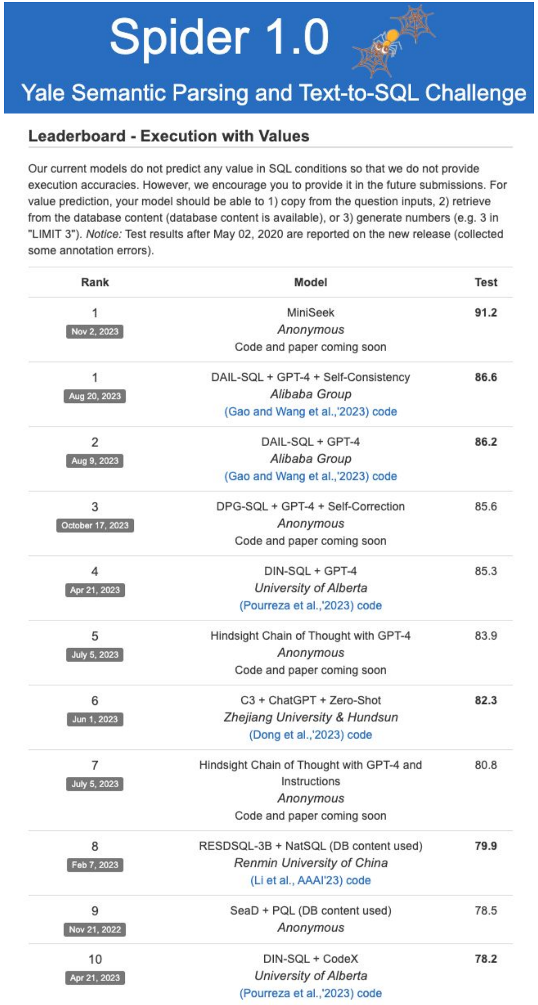
Task Challenges (inherent challenges)

Benchmarks

- Spider 1.0
- BIRD
- Spider 2.0
- BIRD-Interact

Benchmarks

- Spider 1.0
  - Large-scale
  - Cross-domain
  - Complex SQL
- BIRD
- Spider 2.0
- BIRD-Interact



Spider 1.0

Yale Semantic Parsing and Text-to-SQL Challenge

Leaderboard - Execution with Values

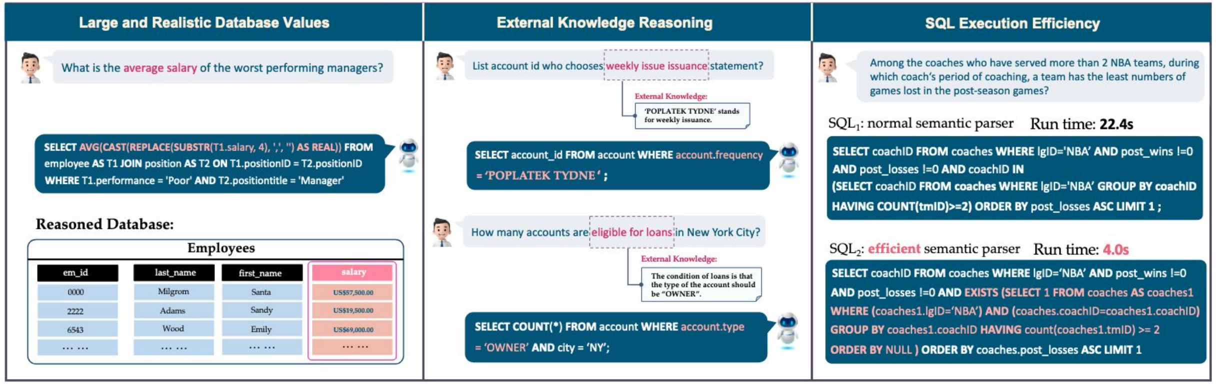
Our current models do not predict any value in SQL conditions so that we do not provide execution accuracies. However, we encourage you to provide it in the future submissions. For value prediction, your model should be able to 1) copy from the question inputs, 2) retrieve from the database content (database content is available), or 3) generate numbers (e.g. 3 in "LIMIT 3"). *Notice:* Test results after May 02, 2020 are reported on the new release (collected some annotation errors).

Rank	Model	Test
1 Nov 2, 2023	MiniSeek *Anonymous* Code and paper coming soon	**91.2**
1 Aug 20, 2023	DAIL-SQL + GPT-4 + Self-Consistency *Alibaba Group* (Gao and Wang et al.,'2023) code	**86.6**
2 Aug 9, 2023	DAIL-SQL + GPT-4 *Alibaba Group* (Gao and Wang et al.,'2023) code	**86.2**
3 October 17, 2023	DPG-SQL + GPT-4 + Self-Correction *Anonymous* Code and paper coming soon	85.6
4 Apr 21, 2023	DIN-SQL + GPT-4 *University of Alberta* (Pourreza et al.,'2023) code	85.3
5 July 5, 2023	Hindsight Chain of Thought with GPT-4 *Anonymous* Code and paper coming soon	83.9
6 Jun 1, 2023	C3 + ChatGPT + Zero-Shot *Zhejiang University & Hundsun* (Dong et al.,'2023) code	**82.3**
7 July 5, 2023	Hindsight Chain of Thought with GPT-4 and Instructions *Anonymous* Code and paper coming soon	80.8
8 Feb 7, 2023	RESDSQL-3B + NatSQL (DB content used) *Renmin University of China* (Li et al., AAAI'23) code	**79.9**
9 Nov 21, 2022	SeaD + PQL (DB content used) *Anonymous*	78.5
10 Apr 21, 2023	DIN-SQL + CodeX *University of Alberta* (Pourreza et al.,'2023) code	**78.2**

Screenshot 2026.03.29

Benchmarks

- Spider
- BIRD
  - Large-scale
  - dirty values
  - external knowledge
  - efficiency
- Spider 2.0
- BIRD-Interact

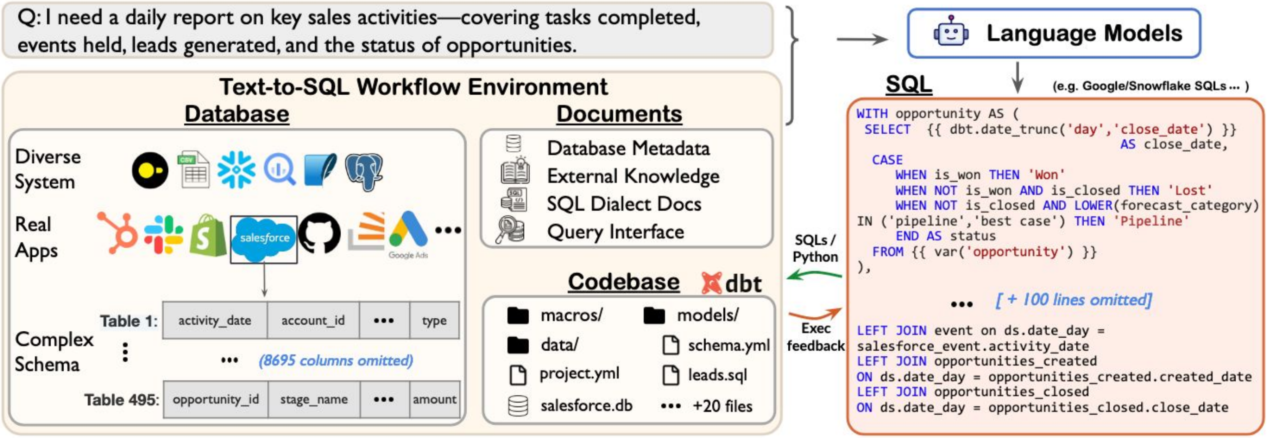


	Model	Code	Size	Oracle Knowledge	Dev (%)	Test (%)
	Human Performance *Data Engineers + DB Students*			✓		**92.96**
1 Dec 16, 2025	AskData + GPT-4o *AT&T CDO - DSAIR* [Shkapenyuk et al. '25]		UNK	✓	77.64	**81.95**
2 Sep 25, 2025	Agentar-Scale-SQL *Ant Group* [Pengfei Wang et al. '25]	[link]	UNK	✓	74.90	81.67
3 July 14, 2025	LongData-SQL *LongShine AI Research*		UNK	✓	74.32	77.53
4 Jan 02, 2026	Zhiwen-Lingsi-Agent *China Telecom, TeleAI*		UNK	✓	73.53	76.63
5 Jan 26, 2026	DeepEye-SQL *Anonymous*		UNK	✓	73.53	76.58
6 Dec 4, 2025	Q-SQL *AWS-Quick Science* [Ravi Shankar et al.'25]		30B-3B-MoE	✓	72.99	76.47
7 Feb 6, 2026	MIC^2-SQL *Hunan University*		UNK	✓	74.45	76.41
8 Jan 14, 2026	SiriusAI-Text2SQL-Agent *Tencent Data Computing Platform*		UNK	✓	74.64	76.30
9 Apr 16, 2025	CHASE-SQL + Gemini *Google Cloud* [Pourreza et al. '24]		UNK	✓	74.90	76.02
10 Feb 21, 2026	RED-SQL *South China Normal University*		30B	✓	74.19	75.91

Screenshot 2026.03.29

Benchmarks

- Spider
- BIRD
- Spider 2.0
  - enterprise workflows
  - External documents
  - large schemas
  - multi-dialect
- BIRD-Interact



Benchmarks

- Spider
- BIRD
- Spider 2.0
  - enterprise workflows
  - long context
  - large schemas
  - multi-dialect
- BIRD-Interact

Spider 2.0

Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

Leaderboard

Spider 2.0-Snow | Spider 2.0-DBT | Spider 2.0-lite

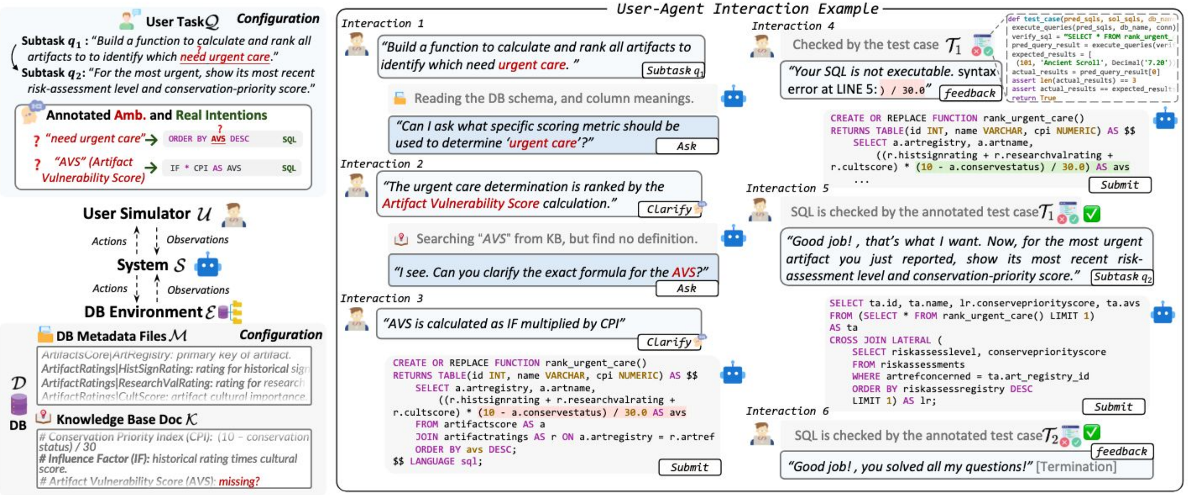
Spider 2.0-Snow is a self-contained text-to-SQL task that includes well-prepared database metadata and documentation, includes **547** examples, all hosted on *Snowflake*, which offers participants free quotas.
Methods with -* use special settings (ground-truth tables) and are not included in the ranking.
Since we continually check the accuracy of the evaluation metrics (while the questions remain fixed), the scores may change slightly over time.

Rank	Method	Score
1 Mar 1, 2026	Genloop's Sentinel Agent v2 Pro *Genloop*	96.70
2 Feb 18, 2026	Native mini *usenative.ai*	96.53
3 Feb 9, 2026	QUVI-3 + Gemini-3-pro-preview *DAQUV*	94.15
4 Feb 3, 2026	TCDataAgent-SQL with Contextual Scaling Engine *Tencent Cloud Big Data*	93.97
5 Jan 27, 2026	Prism Swarm with Deepthink + Claude-Sonnet-4.5 *Paytm*	90.49
6 Feb 17, 2026	Genloop's Sentinel Agent v2 *Genloop*	88.48
7 Feb 6, 2026	QUVI-3 + Claude-Opus-4.6 *DAQUV*	86.28
8 Jan 7, 2026	Ask Data with Relational Knowledge Graph *AT&T CDO & RelationalAI*	86.28
9 Dec 16, 2025	ByteBrain-Agent *ByteDance Infra System Lab*	84.10
10 Feb 13, 2026	Genloop's Sentinel Agent v1.5 *Genloop*	83.36
11 Jan 9, 2026	AiCheng Agent *alibaba_cfo_tech*	82.81
12 Jan 2, 2026	Prism Swarm + Claude-Sonnet-4.5 *Paytm*	82.63
13 Dec 5, 2025	LingXi Agent + Claude-Sonnet-4.5 *Ant Group*	79.89
14 Dec 9, 2025	Arctic-FLEX *Snowflake AI Research*	75.14
15 Nov 24, 2025	Sophon-Agent *ByteDance DataPlatform LLM*	74.04
16 Feb 28, 2026	APEX-SQL *LIGHTSPEED* [Cao et al. '25]	73.13
17 Dec 26, 2025	QiSi-SQL + Deepseek3.2 *Ant Group Tech Risk*	70.38
18 Sep 26, 2025	PExA *Bloomberg - AI Engineering group*	70.20
19 Dec 4, 2025	Chicory AI Agent + Claude Sonnet 4.5 + Opus 4.5 Judge *Chicory AI*	67.28
20 Oct 2, 2025	SSDAT + GPT-5	65.63
21 Dec 7, 2025	DSR-SQL + DeepSeek-R1 *GDUT DMIR Lab* [Hao et al. '25]	63.80

Screenshot 2026.03.29

Benchmarks

- Spider
- BIRD
- Spider 2.0
- BIRD-Interact
  - Multi-turn interaction
  - Ambiguity
  - User simulator
  - Full CRUD tasks
    (Create, Read, Update, and Delete)



Rank	Model	Institute	Link	Date	User Simulator	Efficiency	Success Rate (%)
	Claude-Opus-4.6			2026-02-17	Claude-Haiku-4-5	2.81	**22.17**
	Gemini-2.5-Pro			2025-08-22	GPT-4o	2.71	18.83
	O3-Mini			2025-08-22	GPT-4o	3.01	18.50
4	Qwen-3-Coder-480B			2025-08-22	GPT-4o	5.22	18.50
5	DeepSeek-Chat-V3.1			2025-08-22	GPT-4o	5.88	17.00
6	Kimi-2.5			2026-02-17	Claude-Haiku-4-5	4.78	16.67
7	Claude-Sonnet-4			2025-08-22	GPT-4o	4.61	15.67
8	Qwen3-Coder-30B			2026-02-02	Claude-Haiku-4-5	5.76	14.50
9	GPT-5			2025-08-22	GPT-4o	2.93	13.83
10	Claude-3.7-Sonnet			2025-08-22	GPT-4o	4.57	12.50

Where Are We?

Where Are We?

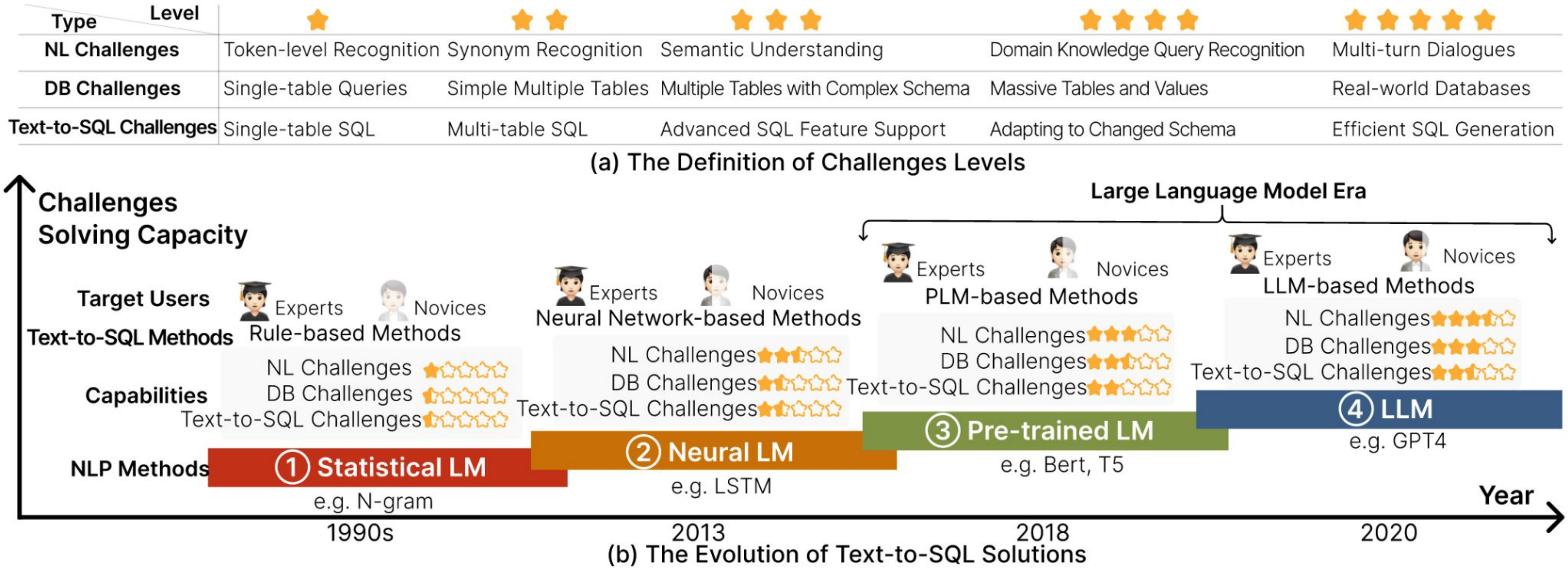
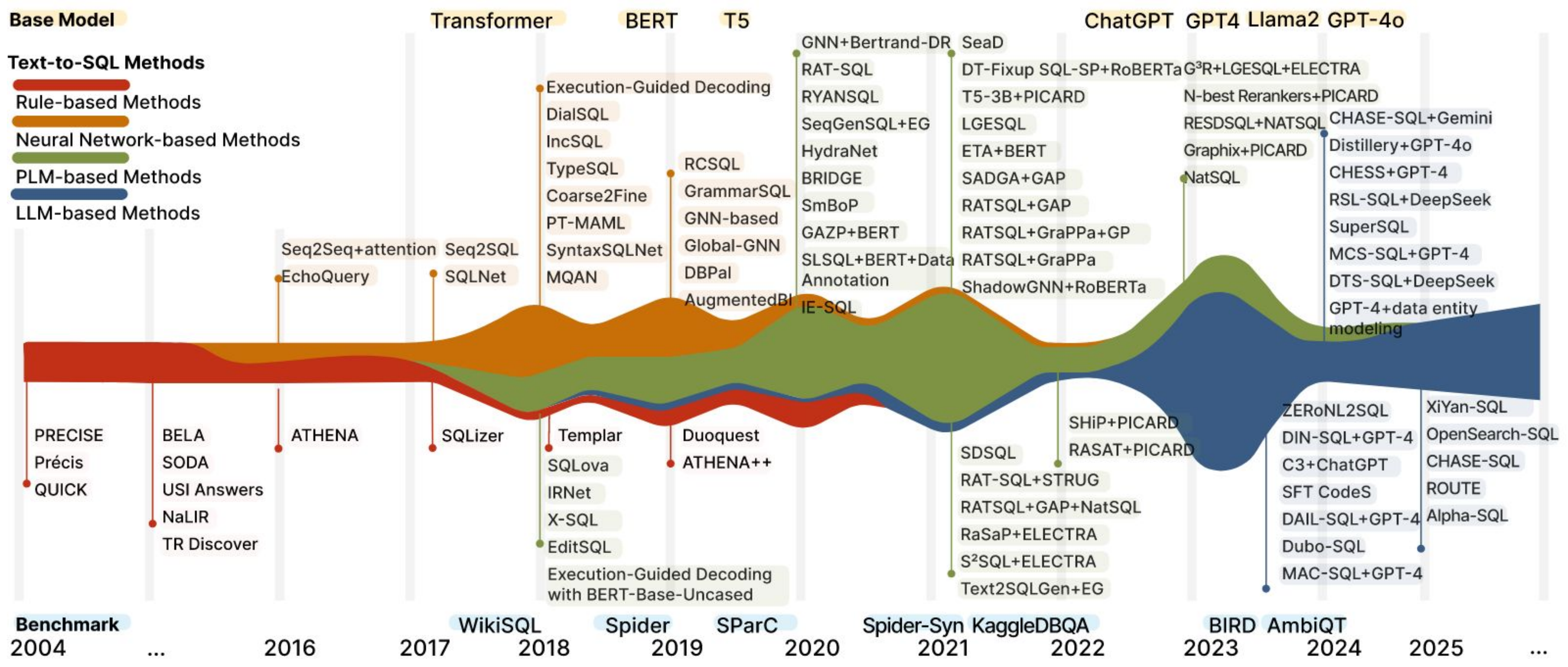


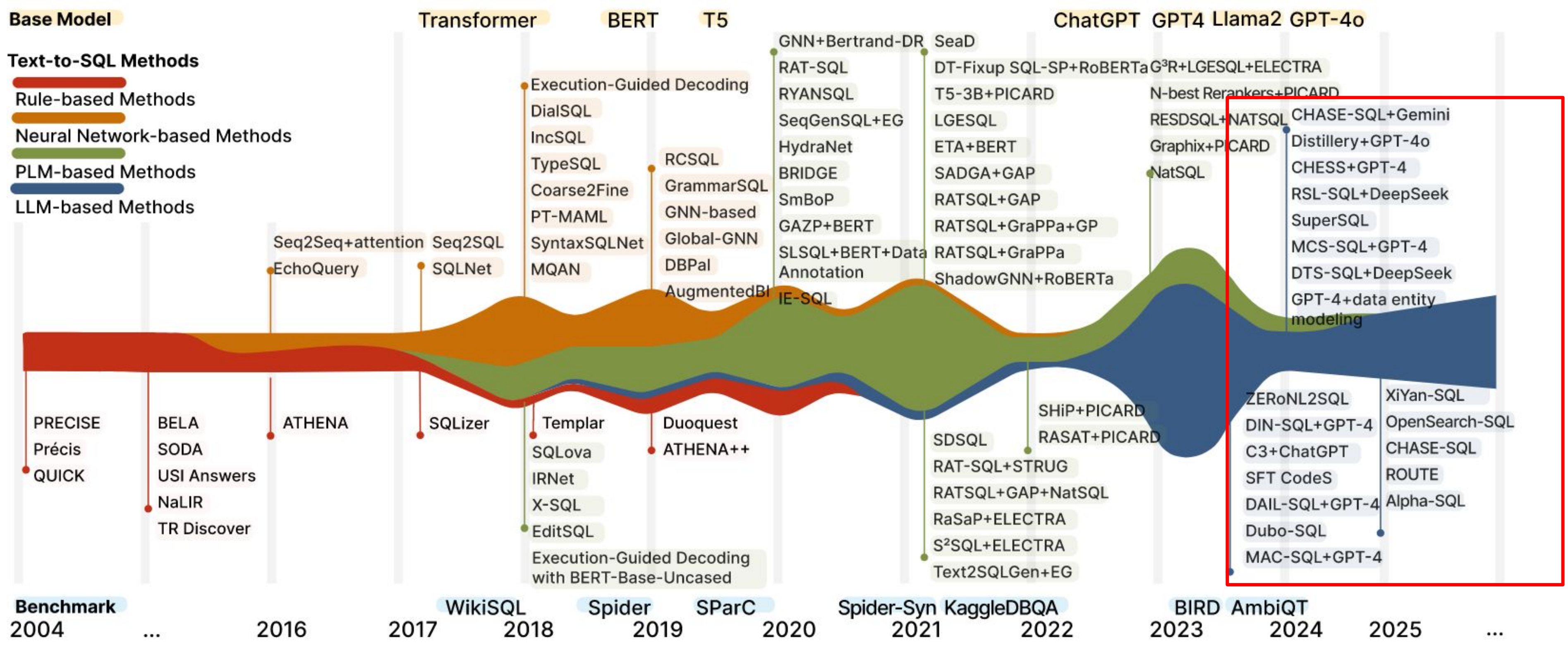
Figure: The Evolution of NL2SQL Solutions from the Perspective of Language Models.

Where Are We?



https://github.com/HKUSTDial/NL2SQL_Handbook

Where Are We?



https://github.com/HKUSTDial/NL2SQL_Handbook

Tutorial Outline

- Problem Definition, Challenges, Benchmarks, Overview
- NL2SQL Solutions with LLM
  - Prompting-based
  - Training-based
- NL2SQL Solutions with Agent
- Open Problem

NL2SQL Solutions with LLMs

Categorization of Existing Studies

- Prompt-based Methods
  - Inference-only
  - No extra tools or environment interaction
- CPT/SFT Training-based Methods
  - Open-source models
  - Training based on pretrained LLM

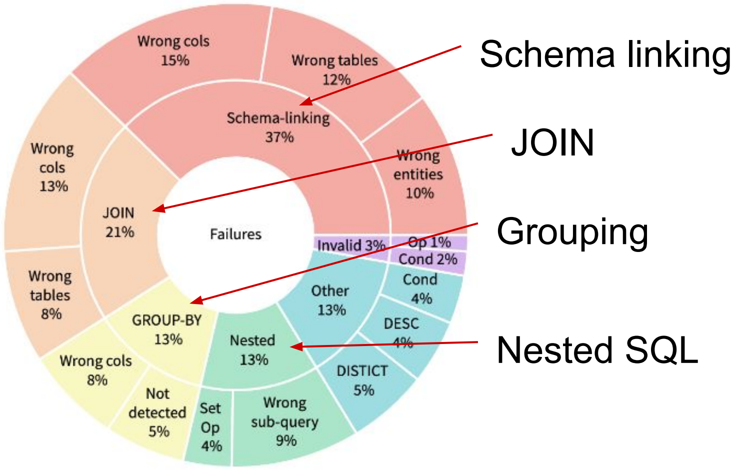
Prompt-based Methods

DIN-SQL (NIPS 2023, Pourreza et al.)

- LLM prompting was still clearly weaker than fine-tuned models on Spider. On Spider dev, the paper reports 67.4 EX for few-shot GPT-4 versus 84.5 EX for fine-tuned RED-SQL + NatSQL.
- Their key hypothesis was: one-shot prompting is too hard for complex SQL, so the task should be broken into smaller steps.

Fine-tuning approaches	
Method	**Execution accuracy**
RED-SQL 3B + NatSQL [Li et al., 2023a]	84.5
T5-3B + PICARD [Scholak et al., 2021]	79.3
Inference-only approaches	
Method	**Execution accuracy**
Zero-shot GPT-4 (Ours)	64.9
Few-shot GPT-4 (Ours)	67.4
Zero-shot CodeX [Rajkumar et al., 2022]	55.1
Few-shot CodeX (Ours)	61.5

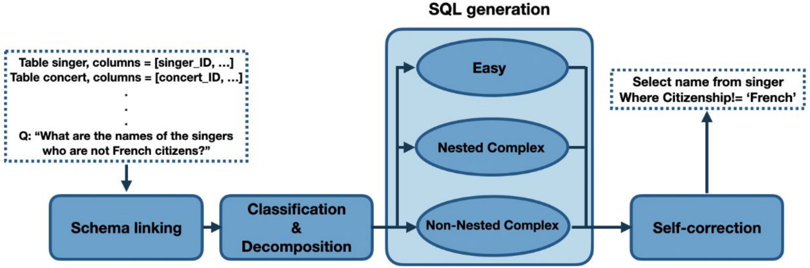
Performance on the dev split of Spider

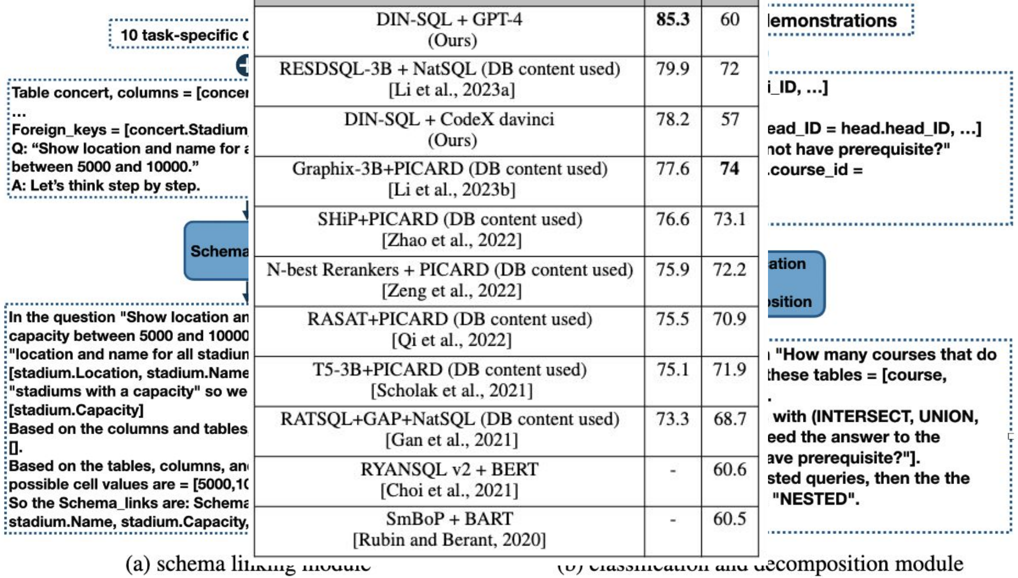


Prompt-based Methods

DIN-SQL (NIPS 2023, Pourreza et al.)

- decomposing a text-to-SQL task consists of four modules (1) schema linking, (2) query classification and decomposition, (3) SQL generation, and (4) self-correction
- Result: the method improves simple few-shot prompting by about 10 points, reaching 85.3 EX on Spider test and 55.9 EX on BIRD test, which was state of the art in execution accuracy at the time.





Model	EX	EM
DIN-SQL + GPT-4 (Ours)	**85.3**	60
RESDSQL-3B + NatSQL (DB content used) [Li et al., 2023a]	79.9	72
DIN-SQL + CodeX davinci (Ours)	78.2	57
Graphix-3B+PICARD (DB content used) [Li et al., 2023b]	77.6	**74**
SHiP+PICARD (DB content used) [Zhao et al., 2022]	76.6	73.1
N-best Rerankers + PICARD (DB content used) [Zeng et al., 2022]	75.9	72.2
RASAT+PICARD (DB content used) [Qi et al., 2022]	75.5	70.9
T5-3B+PICARD (DB content used) [Scholak et al., 2021]	75.1	71.9
RATSQL+GAP+NatSQL (DB content used) [Gan et al., 2021]	73.3	68.7
RYANSQL v2 + BERT [Choi et al., 2021]	-	60.6
SmBoP + BART [Rubin and Berant, 2020]	-	60.5

Prompt-based Methods

DAIL-SQL (VLDB 2023; by Alibaba)

- what parts of prompt design actually matters to text-to-sql performance?
  - Question/Schema representation
    - Basic, Text Representation, OpenAI format, Code Representation (DDL)
  - example selection (few-shot demonstration)
    - Random, Question similarity, Masked question similarity, Query similarity
  - example organization
    - Full information, SQL-only

```
Table continents, columns = [ContId, Continent]
Table countries, columns = [CountryId, CountryName,
↳ Continent]
Q: How many continents are there?
A: SELECT
```

Listing 1: Example of Basic Prompt

```
Given the following database schema:
continents: ContId, Continent
countries: CountryId, CountryName, Continent

Answer the following: How many continents are there?
SELECT
```

Listing 2: Example of Text Representation Prompt

```
### Complete sqlite SQL query only and with no
↳ explanation
### SQLite SQL tables, with their properties:
#
# continents(ContId, Continent)
# countries(CountryId, CountryName, Continent)
#
### How many continents are there?
SELECT
```

Listing 3: Example of OpenAI Demostration Prompt

```
/* Given the following database schema: */
CREATE TABLE continents(
    ContId int primary key,
    Continent text,
    foreign key(ContId) references countries(Continent)
);

CREATE TABLE countries(
    CountryId int primary key,
    CountryName text,
    Continent int,
    foreign key(Continent) references continents(ContId)
);

/* Answer the following: How many continents are there?
↳ */
SELECT
```

Listing 4: Example of Code Representation Prompt

Gao, Dawei, et al. "Text-to-sql empowered by large language models: A benchmark evaluation." PVLDB, 17(5): 1132-1145, 2024.

Prompt-based Methods

DAIL-SQL (VLDB 2023; by Alibaba)

- what parts of prompt design actually matters to text-to-sql performance?
  - Question/Schema representation
    - Basic, Text Representation, OpenAI format, Code Representation (DDL)
  - example selection (few-shot demonstration)
    - Random, Question similarity, Masked question similarity, Query similarity
  - example organization
    - Full information, SQL-only

```
/* Given the following database schema: */
${DATABASE_SCHEMA}
/* Answer the following: How many authors are there? */
SELECT count(*) FROM authors

/* Given the following database schema: */
${DATABASE_SCHEMA}
/* Answer the following: How many farms are there? */
SELECT count(*) FROM farm

${TARGET_QUESTION}
```

Listing 6: Example of Full-Informati

```
/* Some SQL examples are provided based on similar
↳ problems: */
SELECT count(*) FROM authors

SELECT count(*) FROM farm

${TARGET_QUESTION}
```

Listing 7: Example of SQL-Only Organization.

```
/* Some example questions and corresponding SQL queries
↳ are provided based on similar problems: */
/* Answer the following: How many authors are there? */
SELECT count(*) FROM authors

/* Answer the following: How many farms are there?. */
SELECT count(*) FROM farm

${TARGET_QUESTION}
```

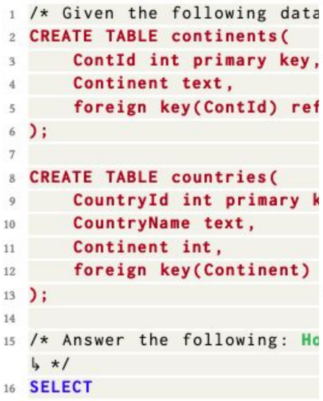
Listing 8: Example of DAIL Organization.

Gao, Dawei, et al. "Text-to-sql empowered by large language models: A benchmark evaluation." PVLDB, 17(5): 1132-1145, 2024.

Prompt-based Methods

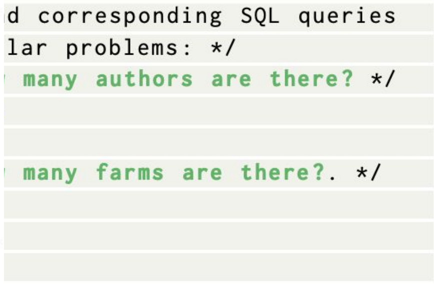
DAIL-SQL (VLDB 2023; by Alibaba)

- Question/Schema Representation: Code Representation
- Example Selection: combining masked question similarity and SQL query similarity
- Example Organization: only preserving question-to-SQL mappings while removing token-expensive database schema from examples
- DAIL-SQL reached 86.6% execution accuracy on Spider test, surpassing DIN-SQL by 1.3% with much lower token cost.



Schema Re

Classification	Method	Dev. EX (%)	Test EX (%)
rule-based	Duoquest [4]	63.5	63.5
	ATHENA++ [44]	78.82	-
PLM-based	ValueNet [5]	67.0	-
	BRIDGE v2 + BERT [25]	68.0	64.3
	T5-Base [43]	57.9	-
	T5-Large [43]	67.2	-
	T5-3B [43]	74.4	70.1
	T5-3B + PICARD [43]	79.3	75.1
	RESDSQL-3B + NatSQL [23]	84.1	79.9
	Fine-tuned BERT (ours)	53.6	-
LLM-based	C3 + ChatGPT + Zero-Shot [13]	81.8	82.3
	DIN-SQL + GPT-4 [37]	82.8	85.3
	DAIL-SQL + GPT-4	83.1	86.2
	DAIL-SQL + GPT-4 + Self-consistency	83.6	**86.6**



Organization

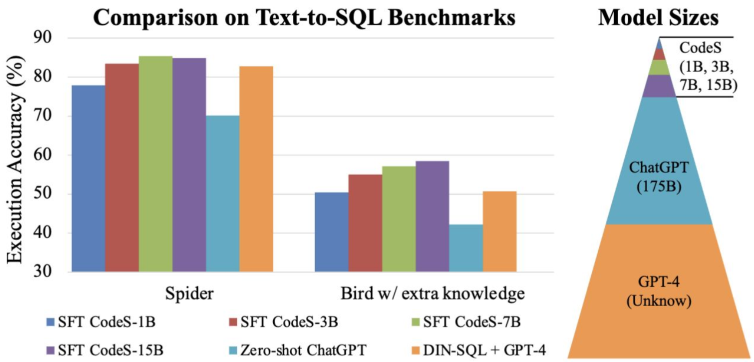
Gao, Dawei, et al. "Text-to-sql empowered by large language models: A benchmark evaluation." PVLDB, 17(5): 1132-1145, 2024.

Training LLMs for NL2SQL

Training LLMs for NL2SQL

CodeS (VLDB 2023, Alibaba Group)

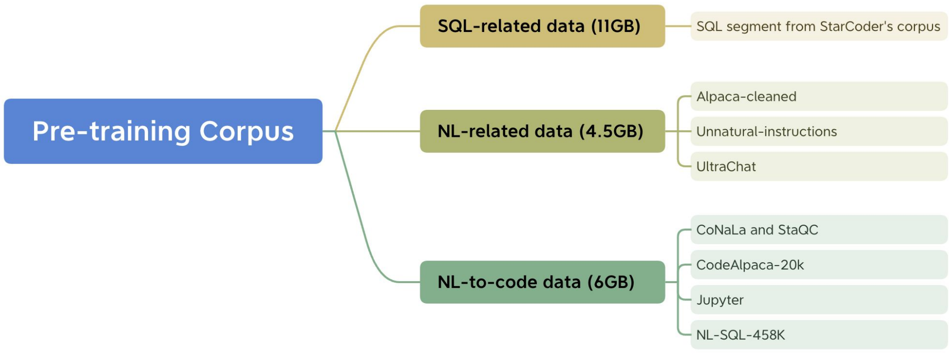
- Motivation: strong text-to-SQL systems were increasingly based on closed-source LLMs, which raise issues of privacy, controllability, and cost.
- Solution Overview:
  - CodeS introduces a series of open-source language models (ranging from 1B to 15B parameters) specifically tailored for text-to-SQL tasks
  - curate 21.5GB SQL-centric corpus for SQL-centric incremental pre-training
  - bi-directional data augmentation for new domains SFT



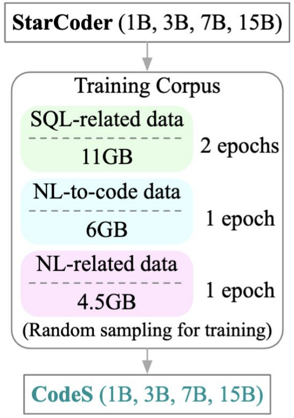
Li, Haoyang, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. "Codes: Towards building open-source language models for text-to-sql." Proceedings of the ACM on Management of Data 2, no. 3 (2024): 1-28.

Data Collection for CPT

- 21.5GB CPT corpus: 11GB SQL-related data, 6GB NL-to-code data, and 4.5GB NL-related data
- Enhanced capabilities: improvements in both SQL generation and natural language understanding
- Training recipe:
  - start from StarCoder
  - do incremental pre-training
  - then supervised fine-tuning target domain



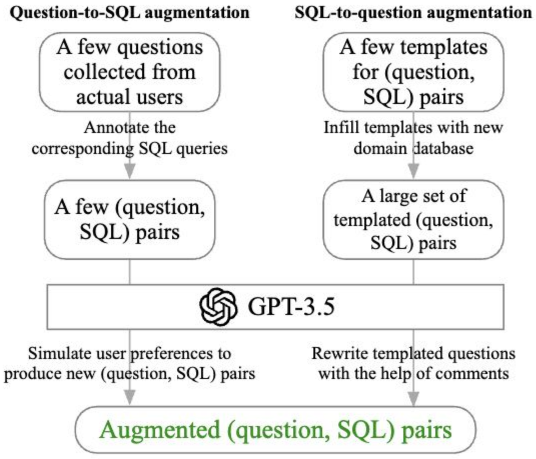
Step 1: Collect SQL-related corpus



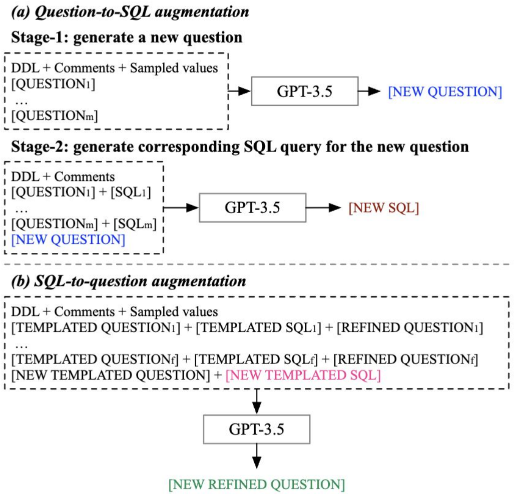
Step 2: Incremental pre-training

Data Augmentation for SFT

- Bi-Directional Data Augmentation for New Domains
  - Question-to-SQL: starting from real user questions, manually annotate, and expanding using GPT-3.5
  - SQL-to-Question: leveraging Spider-style templates, populating with new domain schemas, and refining via GPT-3.5
- Enhanced capabilities: rapid domain adaptation with minimal annotation effort



Bi-directional augmentation



Prompt formats used in data augmentation.

Achieve SOTA Performance

Table 5: Evaluation of SFT CodeS on Spider's dev set.

Methods	EX%	TS%
Fine-tuning-based methods		
T5-3B + PICARD [57]	79.3	69.4
RESDSQL-3B + NatSQL [36]	84.1	73.5
Graphix-T5-3B + PICARD [37]	81.0	75.0
Fine-tuned SQL-PaLM [61]	82.8	78.2
SFT Llama2-7B [65]	77.8	73.0
SFT Llama2-13B [65]	81.6	76.6
Prompting-based methods		
GPT-4 (few-shot) [51]	76.8	67.4
C3 + ChatGPT [16]	81.8	71.4
Self-Debug + Codex davinci [9]	84.1	-
DIN-SQL + GPT-4 [51]	82.8	74.2
DAIL-SQL + GPT-4 [21]	83.1	76.6
SQL-PaLM (few-shot) [61]	82.7	77.3
DAIL-SQL + GPT-4 + Self-Consistency [21]	83.6	76.2
Ours		
SFT CodeS-1B	77.9	72.2
SFT CodeS-3B	83.4	78.1
SFT CodeS-7B	**85.4**	**80.3**
SFT CodeS-15B	84.9	79.4

Table 6: Evaluation of SFT CodeS on BIRD's dev/test sets.

	Dev		Dev w/ EK		Test		Test w/ EK	
Methods	EX%	VES%	EX%	VES%	EX%	VES%	EX%	VES%
Baseline methods								
Fine-tuned T5-3B [38]	10.37	13.62	23.34	25.57	11.17	15.17	24.05	27.80
PaLM-2 [38]	-	-	27.38	-	-	-	33.04	-
Codex 175B [38]	25.42	33.37	34.35	43.41	24.86	35.40	36.47	41.60
ChatGPT [38]	24.05	27.97	42.24	-	26.77	36.68	39.30	51.40
ChatGPT + CoT [38]	25.88	32.33	36.64	42.30	28.95	49.69	40.08	56.56
Claude-2 [38]	-	-	42.70	-	-	-	49.02	-
GPT-4 [38]	-	-	49.15	-	-	-	54.89	60.77
DIN-SQL + GPT-4 [51]	-	-	50.72	58.79	-	-	55.90	59.44
DAIL-SQL + GPT-4 [21]	-	-	54.76	56.08	-	-	57.41	61.95
SFT Llama2-7B [65]	35.53	36.09	45.37	46.98	-	-	-	-
SFT Llama2-13B [65]	41.85	44.00	53.91	58.77	-	-	-	-
Ours								
SFT CodeS-1B	38.46	41.77	50.46	51.07	-	-	-	-
SFT CodeS-3B	43.42	44.55	55.02	56.54	-	-	-	-
SFT CodeS-7B	45.24	48.13	57.17	58.80	50.25	54.84	59.25	63.62
SFT CodeS-15B	**47.91**	**49.60**	**58.47**	**59.87**	**52.15**	**56.99**	**60.37**	**64.22**

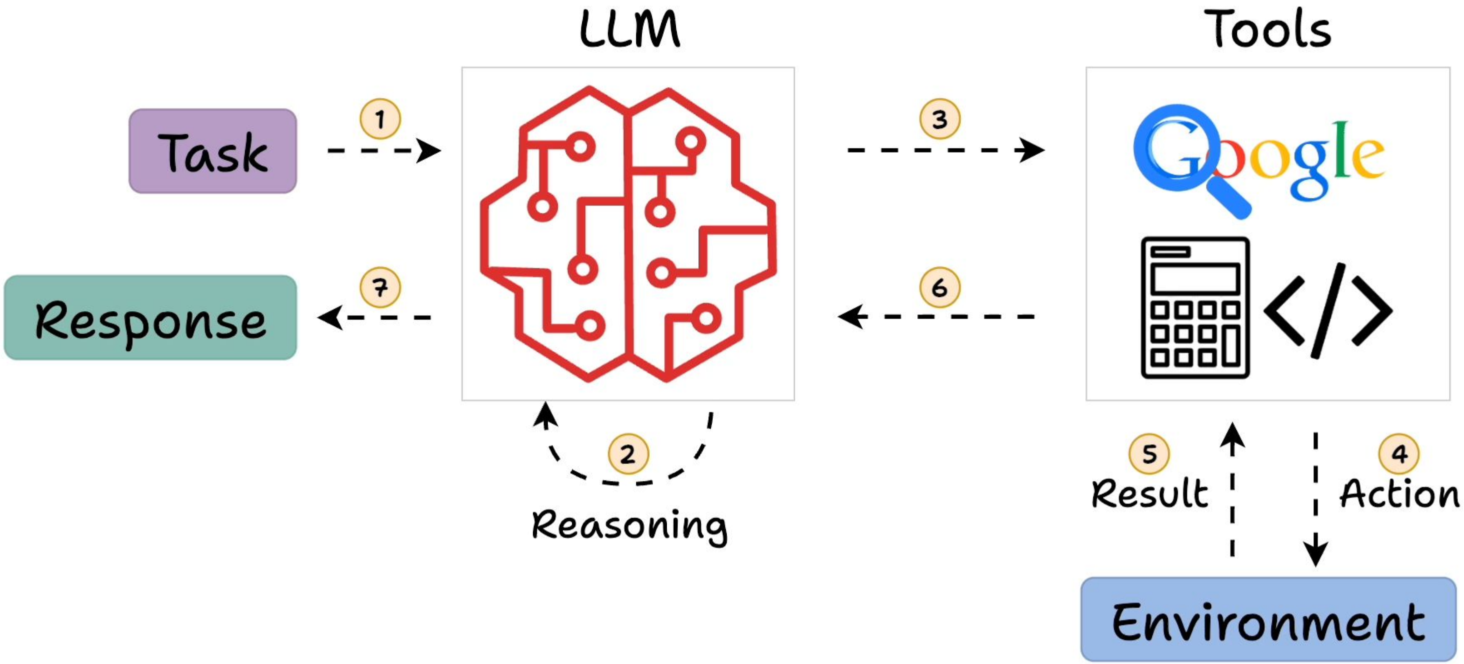
Takeaways

- Prompt-based methods showed that large gains can come from better inference-time reasoning design: decomposition, demonstration selection, and self-correction all matter a lot for text-to-SQL.
- Training-based methods pushed this further by internalizing SQL-specific skills into the model through SQL-centric pretraining and supervised fine-tuning.
- Remaining gap: They still do not fully solve large-schema grounding, external knowledge, and long workflow execution, motivating agentic methods.

Tutorial Outline

- Problem Definition, Challenges, Overview
- NL2SQL Solutions with LLM
- NL2SQL Solutions with Agent
- Open Problem

What is the (Reasoning) Agent?



https://blog.dailydoseofds.com/p/intro-to-react-reasoning-and-action

NL2SQL Solutions with Agents

- CHASE-SQL (ICLR 2025, Google Cloud and Stanford)
  - how to reliably get and select the correct SQL
  - candidate generation with execution feedback
  - Value Retrieval: Utilizes the MinHash LSH to search for values related to the user query
  - Employs an SQL selection agent fine-tuned specifically for the database to select the final SQL from multiple candidates.

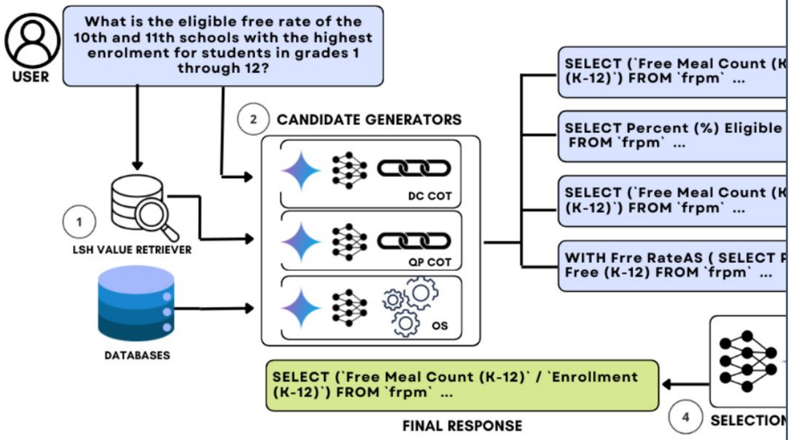
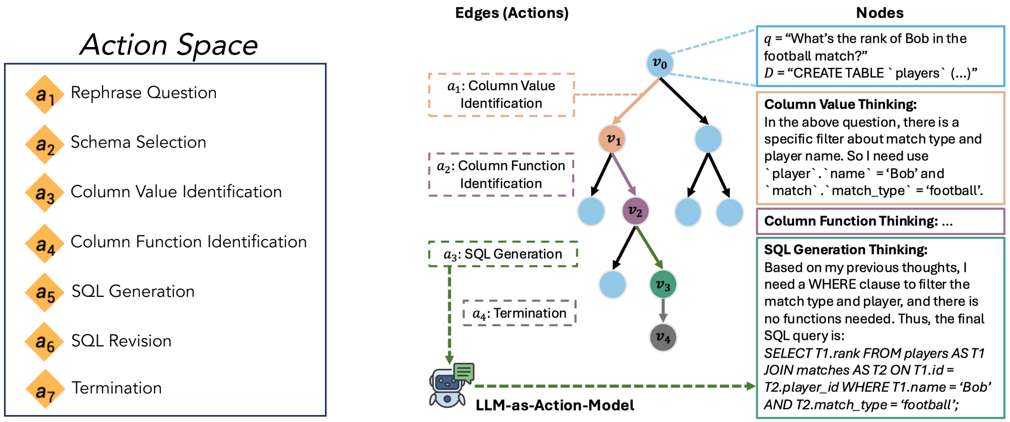


Table 2: Performance Comparison of different Text-to-SQL methods on BIRD benchmark.

Method	EX (Dev)	EX (Test)
Published		
CHASE-SQL + Gemini 1.5 (Ours)	**73.01**	**73.0**
CHASE-SQL + Claude 3.5 Sonnet (Ours)	**69.53**	–
CHASE-SQL + Mistral Large (Ours)	**70.33**	–
Distillery + GPT-4o (Maamari et al., 2024)	67.21	71.83
CHESS (Talaei et al., 2024)	65.00	66.69
MCS-SQL + GPT-4 (Lee et al., 2024)	63.36	65.45
SuperSQL (Li et al., 2024a)	58.5	62.66
Undisclosed		
Insights AI	72.16	70.26
AskData + GPT-4o	72.03	72.39
OpenSearch-v2 + GPT-4o	69.3	72.28
PURPLE-RED + GPT-4o	68.12	70.21
Arcwise + GPT-4o	67.99	66.21
ExSL + granite-34b-code	67.47	67.76

NL2SQL Solutions with Agents

- **Alpha-SQL** (ICML 2025, UCSD and snowflake)
  - text-to-SQL should not be treated as a single generation step, but instead cast it as a structured search problem.
  - Alpha-SQL then uses Monte Carlo Tree Search (MCTS) to explore that space progressively, so the SQL is built step by step rather than guessed all at once.



Tree-based Search:

- Each *edge* corresponds to an *agentic action* in the query construction process,
- Each *node* represents a *reasoning state* at a specific step, and
- Each *path* corresponds *to a sequence of SQL construction actions* for Text-to-SQL task.

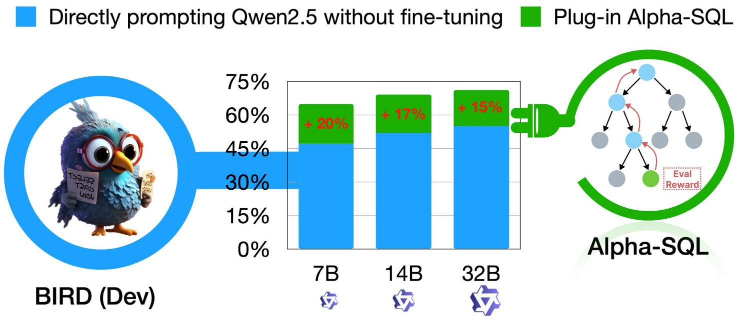
Alpha-SQL: Plug-and-Play Capabilities

Table 2. Execution Accuracy on BIRD Development Dataset.

Method	Inference Model	Selection Model	Zero-shot Setting	Accuracy (%) Simple	Moderate	Challenging	All
SFT CodeS (Li et al., 2024b)	CodeS-7B	-	✗	64.6	46.9	40.3	57.0
SFT CodeS (Li et al., 2024b)	CodeS-15B	-	✗	65.8	48.8	42.4	58.5
Distillery (Maamari et al., 2024)	GPT-4o	-	✗	-	-	-	67.2
CHESS-SQL (Talaei et al., 2024)	Deepseek-Coder-33B	GPT-4-Turbo	✗	-	-	-	65.0
CHESS-SQL (Talaei et al., 2024)	Deepseek-Coder-33B	LLaMA3-70B	✗	-	-	-	61.5
CHASE-SQL (Pourreza et al., 2025)	Gemini-1.5-Pro	Gemini-1.5-Flash	✗	-	-	-	73.0
XiYan-SQL (Gao et al., 2024b)	?	?	✗	-	-	-	73.3
XiYan-SQL (Gao et al., 2024b)	Qwen2.5-Coder-32B	Qwen2.5-Coder-32B	✗	-	-	-	67.0
DAIL-SQL (Gao et al., 2024a)	GPT-4	SC Selection	✓	63.0	45.6	43.1	55.9
SuperSQL (Li et al., 2024a)	GPT-4	SC Selection	✓	66.9	46.5	43.8	58.5
MCS-SQL (Lee et al., 2025)	GPT-4	GPT-4	✓	-	-	-	64.4
RSL-SQL (Cao et al., 2024)	GPT-4o	GPT-4o	✓	74.4	57.1	53.8	67.2
Alpha-SQL (Ours)	**Qwen2.5-Coder-7B**	**SC Selection**	✓	**72.6**	**59.3**	**53.1**	**66.8**
Alpha-SQL (Ours)	**Qwen2.5-Coder-14B**	**SC Selection**	✓	**74.6**	**61.0**	**55.9**	**68.7**
Alpha-SQL (Ours)	**Qwen2.5-Coder-32B**	**SC Selection**	✓	**74.5**	**64.0**	**57.2**	**69.7**

Table 3. Execution Accuracy on Spider Development Dataset.

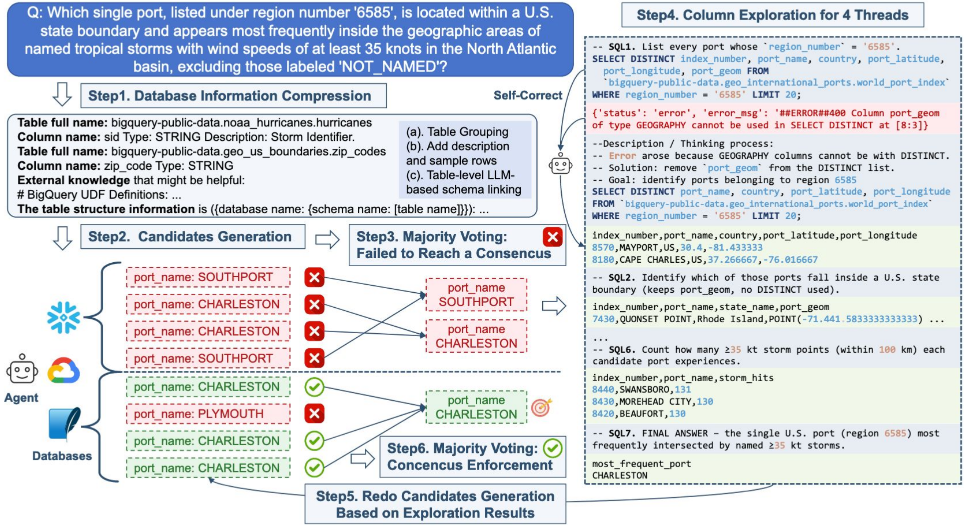
Method	Inference Model	Selection Model	Zero-shot Setting	Accuracy (%) Easy	Medium	Hard	Extra Hard	All
SFT CodeS (Li et al., 2024b)	CodeS-7B	-	✗	94.8	91.0	75.3	66.9	85.4
SFT CodeS (Li et al., 2024b)	CodeS-15B	-	✗	95.6	90.4	78.2	61.4	84.9
C3-SQL (Dong et al., 2023)	GPT-3.5-Turbo	SC Selection	✓	92.7	85.2	77.6	62.0	82.0
DIN-SQL (Pourreza & Rafiei, 2023)	GPT-4	-	✓	92.3	87.4	76.4	62.7	82.8
DAIL-SQL (Gao et al., 2024a)	GPT-4	SC Selection	✓	91.5	90.1	75.3	62.7	83.6
ZeroNL2SQL (Fan et al., 2024)	GPT-4	-	✓	-	-	-	-	84.0
MAC-SQL (Wang et al., 2025)	GPT-4	-	✓	-	-	-	-	86.8
SuperSQL (Li et al., 2024a)	GPT-4	SC Selection	✓	94.4	91.3	83.3	68.7	87.0
Alpha-SQL (Ours)	**Qwen2.5-Coder-7B**	**SC Selection**	✓	**94.0**	**89.2**	**76.4**	**63.3**	**84.0**
Alpha-SQL (Ours)	**Qwen2.5-Coder-14B**	**SC Selection**	✓	**94.0**	**91.0**	**79.9**	**72.3**	**87.0**



NL2SQL Solutions with Agents

ReFoRCE (ICLR 2025 VerifAI Workshop, UCSD and snowflake)

- More realistic problem: large schema (1000+ columns); multiple SQL dialects, ambiguous questions, and long-context metadata;
- ReFoRCE treats text-to-SQL as a modular agent workflow with four stages: Database information compression, Candidate generation, Majority Voting, Column exploration
- SOTA results on Spider 2.0, with 35.83 on Spider 2.0-Snow and 36.56 on Spider 2.0-Lite



Takeaways

- From the agent perspective, natural language to SQL is no longer treated as a single-step generation task, but as a multi-step interaction problem.
- The key insight is that strong performance often requires the system to retrieve context, explore alternatives, validate outputs, and refine after feedback.

Tutorial Outline

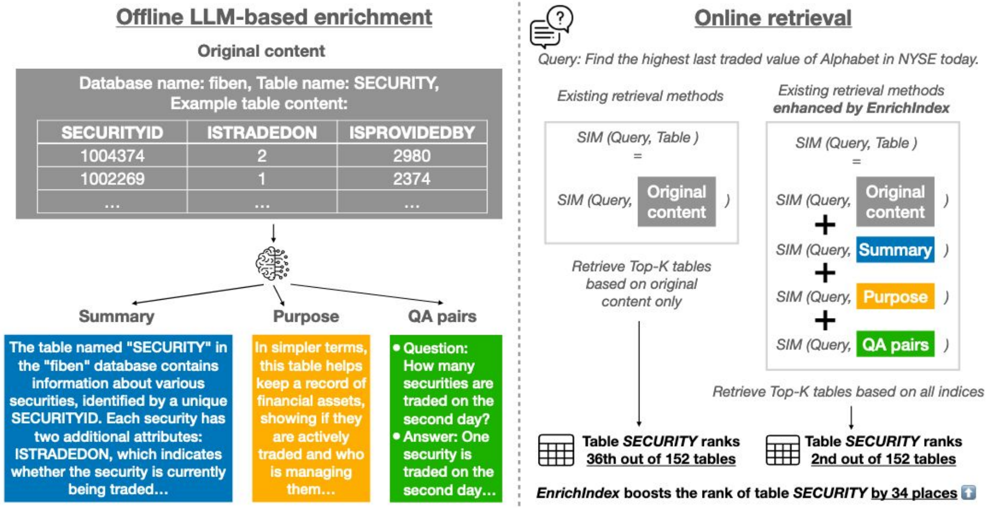
- Problem Definition, Challenges, Overview
- NL2SQL Solutions with LLM
- NL2SQL Solutions with Agent
- Open Problem
  - Knowledgebase-augmented agent
  - Business-realistic evaluation

Knowledgebase-augmented agent

- Core idea:
  - **Build** and **reuse** an external **knowledge base** while interacting with the database, instead of treating every question as an isolated task.
  - Use knowledge to guide future reasoning and reduce repeated mistakes.
- What the knowledgebase can contain:
  - schema structure and semantics,
  - question,SQL demonstrations,
  - domain-specific facts,
  - prior execution traces
  - corrective knowledge learned from earlier failures.

Knowledgebase-augmented agent

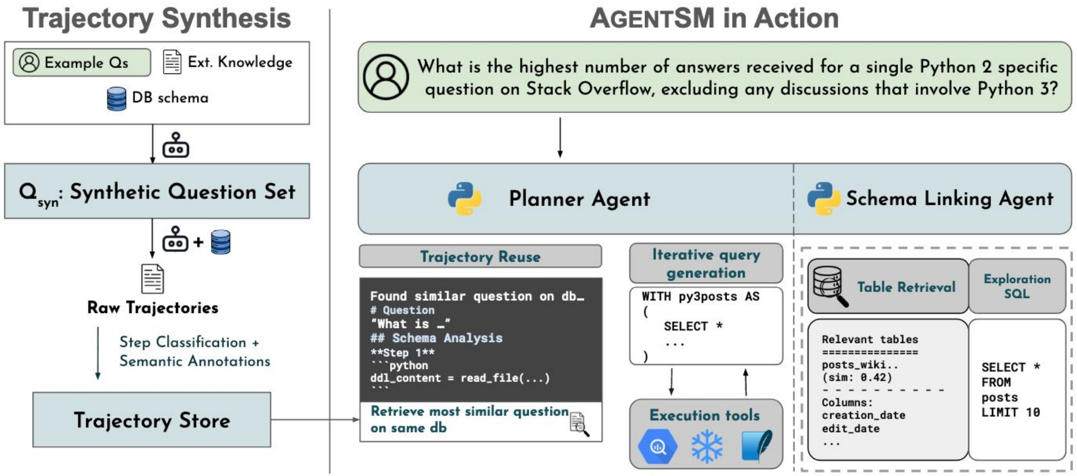
- **EnrichIndex (COLM 2025; MIT, University of Pennsylvania)**
  - semantically-enriched retrieval indices (summary, Purpose, QA pairs)
  - Improve 10% retrieval recall on Spider2.0



Chen, P.B. et al. (2025) “EnrichIndex: Using LLMs to Enrich Retrieval Indices Offline.” arXiv. Available at: https://doi.org/10.48550/arXiv.2504.03598.

Knowledgebase-augmented agent

- **AgentSM (Arxiv 2026.01; Amazon, UC Berkeley, etc.)**
  - Synthesize reasoning trajectory as knowledge
  - Reuse previous trajectories for future reasoning
  - avoid redundant exploration, shorten trajectories, improve reasoning consistency
  - **SOTA 44.8%** EX on Spider 2.0 Lite, reduce token usage by **25%** and trajectory length by **35%.**

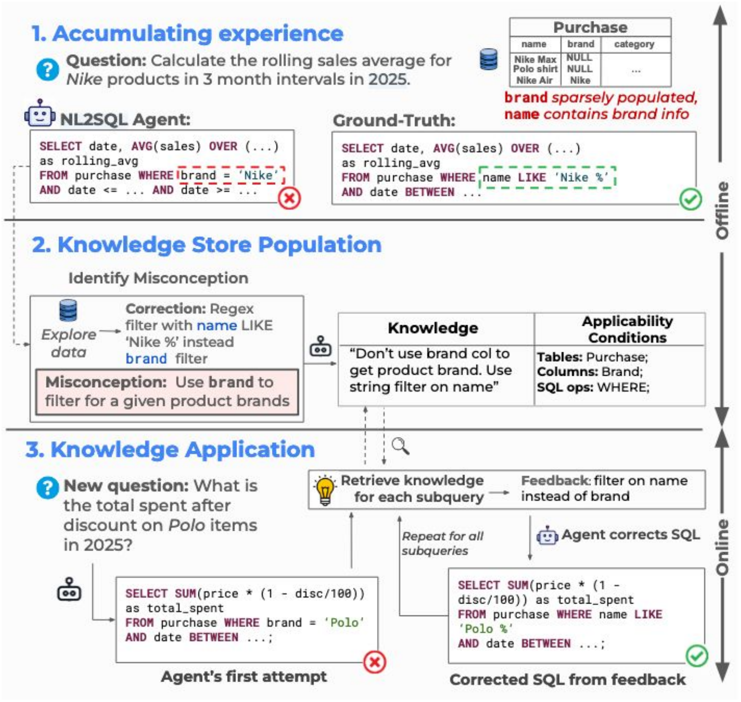


Biswal, A. *et al.* (2026) "AgentSM: Semantic Memory for Agentic Text-to-SQL." arXiv. Available at: https://doi.org/10.48550/arXiv.2601.15709.

Knowledgebase-augmented agent

TK-BOOST (Arxiv 2026.02; UC Berkeley)

- **tribal knowledge**: reusable natural-language corrections learned from the agent's past mistakes on the same database.
- How it works
  - Offline: let the agent answer a few labeled queries, analyze wrong SQL, identify the root misconception, and generate corrective knowledge.
  - Indexing: store each knowledge item with applicability conditions such as relevant tables, columns, and SQL operations.
  - Online: for a new query, first generate a draft SQL, then retrieve applicable knowledge and feed back targeted corrections to repair the SQL.

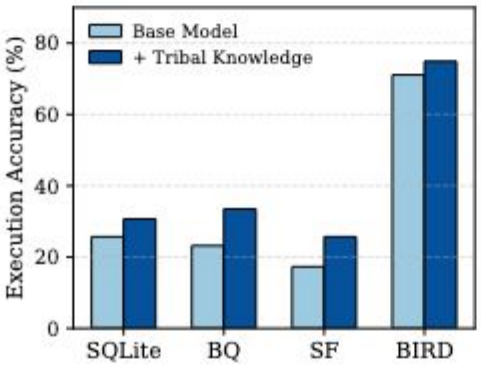


Agarwal, S. *et al.* (2026) "Arming Data Agents with Tribal Knowledge." arXiv. Available at: https://doi.org/10.48550/arXiv.2602.13521.

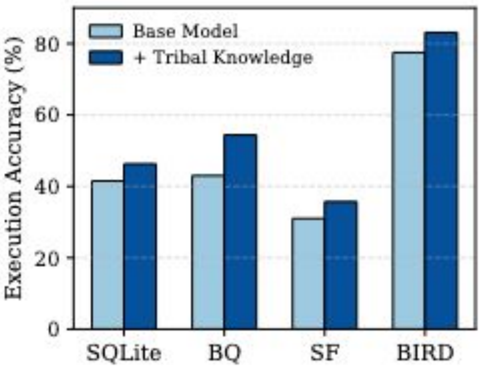
Knowledgebase-augmented agent

TK-BOOST (Arxiv 2026.02; UC Berkeley)

- Tk-Boost is a bolt-on solution that can be applied to any NL2SQL agent to improve the accuracy.
- For ReFoRCE, TK-BOOST raises execution accuracy from 41.5%→46.3% on Spider 2.0 SQLite, 43.0%→54.4% on BigQuery, and 31.0%→35.7% on Snowflake, and 77.5%→83.1% on BIRD.



(a) Agentar-Scale-SQL (32B)



(b) ReFoRCE (GPT-4.1)

Agarwal, S. *et al.* (2026) "Arming Data Agents with Tribal Knowledge." arXiv. Available at: https://doi.org/10.48550/arXiv.2602.13521.

Takeaways

- A reusable knowledge base helps the agent carry forward what it has learned from prior interactions, instead of solving every query from scratch.
- The broader lesson is that stronger text-to-SQL agents not only need better reasoning, but also better memory and reusable knowledge about the environment.
- The field is still under-explored. There is still no perfect knowledge base structure for NL2SQL task.

Business-realistic evaluation

- Current benchmarks are much more realistic than before, but still not enough for business domain enterprise evaluation.
  - BIRD adds dirty values, external knowledge, and efficiency.
  - Spider 2.0 moves to enterprise workflows, large schemas, docs, multi-dialects
  - BIRD-INTERACT evaluates multiturn interactions.
- Text-to-SQL agents must be validated on the target private business database before deployment, but collecting such evaluation data is hard.
- how can we evaluate Text-to-SQL agents in private business scenarios?
  - Synthetic evaluation data
    - Realistic, reflect real business usage

Business-realistic evaluation

- **Business-logic driven data generation** (under submission;)
  - We propose a data synthesis framework that incorporates business logic into data generation pipeline to improve the business realism
  - Our framework consists of 3 stages:
    - Business Logic Modeling
    - Business Logic-Driven Schema Selection
    - Text-to-SQL Data Generation
  - We model the business logic as a 3-layer hierarchical structure
    - Persona
    - Work scenario
    - Workflow

Liu. *et al.* (2026) “Business Logic-Driven Text-to-SQL Data Synthesis for Business Intelligence.” arXiv. Available at: https://arxiv.org/pdf/2601.14518.

Business Logic Modeling

3-layer hierarchical structure:

- Persona (job role)
- Work scenario
- Workflow

Persona

Job Title: Sales Development Manager
Department: Sales Development & Prospecting
Seniority: Manager
Goals
• Deliver pipeline targets
• Improve MQL→SQL conversion
• Maintain SDR productivity
Pain Points
• Volatile lead volume
• Data quality gaps
• AE handoff friction

Business Logic Modeling

3-layer hierarchical structure:

- Persona (job role)
- Work scenario
- Workflow

Persona

Job Title: Sales Development Manager
Department: Sales Development & Prospecting
Seniority: Manager
Goals
- Deliver pipeline targets
- Improve MQL→SQL conversion
- Maintain SDR productivity

Pain Points
- Volatile lead volume
- Data quality gaps
- AE handoff friction

Work Scenario 1

Daily Pipeline Pacing & Rep Productivity Review

Details: Reviews CRM dashboards for SQLs per rep, meetings, and conversion rates versus plan, identifies outliers, and initiates coaching or activity adjustments as needed.

...

Work Scenario n

Funnel Conversion & Sequence Performance Analysis

Details: Analyzes MQL-to-SQL and SQL-to-opportunity conversion by source and segment, identifies drop-offs, compares A/B results, and optimizes sequences, messaging, and lead scoring.

Business Logic Modeling

3-layer hierarchical structure:

- Persona (job role)
- Work scenario
- Workflow

Business Logic Modeling

3-layer hierarchical structure:

- Persona (job role)
- Work scenario
- Workflow

Persona

Job Title: Sales Development Manager
Department: Sales Development & Prospecting
Seniority: Manager
Goals
- Deliver pipeline targets
- Improve MQL→SQL conversion
- Maintain SDR productivity

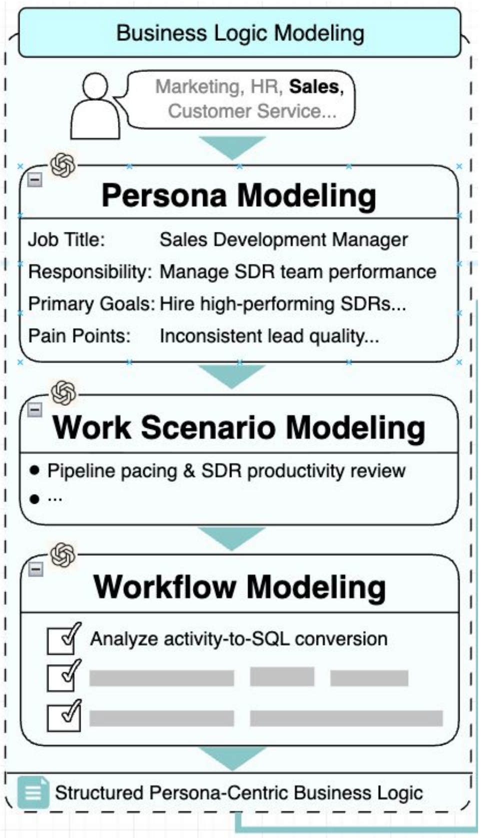
Pain Points
- Volatile lead volume
- Data quality gaps
- AE handoff friction

Work Scenario
Daily Pipeline Pacing & Rep Productivity Review

Workflow (Tasks)
- Set dashboard filters & validate freshness
- Review team pacing vs plan
- Identify rep-level outliers
- Analyze activity→SQL conversion
- Check data quality & routing issues
- Decide coaching / list / sequence actions
- Update forecast & communicate plan

Representative KPIs
- SQLs per Rep
- Pipeline Created ($)
- MQL→SQL Conversion
- Activity→SQL Conversion
- AE SQL Acceptance Rate

Demo business logic



Business-realistic evaluation

- **Business-logic driven data generation** (under submission; CU)
  - LLM-as-a-Judge Metrics:
    - Question-sql alignment
    - Question realism

Method	Evaluation LLM	Question-SQL Alignment(%)	Question Realism(%)
OmniSQL (Li et al., 2025a)	Gemini3 Pro	95.47 (↓ 3.12)	78.91 (↓ 19.53)
	Claude Sonnet 4.5	88.59 (↓ 2.35)	80.94 (↓ 16.40)
SQL-Factory (Li et al., 2025b)	Gemini3 Pro	90.40 (↓ 8.19)	43.75 (↓ 54.69)
	Claude Sonnet 4.5	84.15 (↓ 6.79)	45.43 (↓ 51.91)
Ours	Gemini3 Pro	**98.59**	**98.44**
	Claude Sonnet 4.5	**90.94**	**97.34**

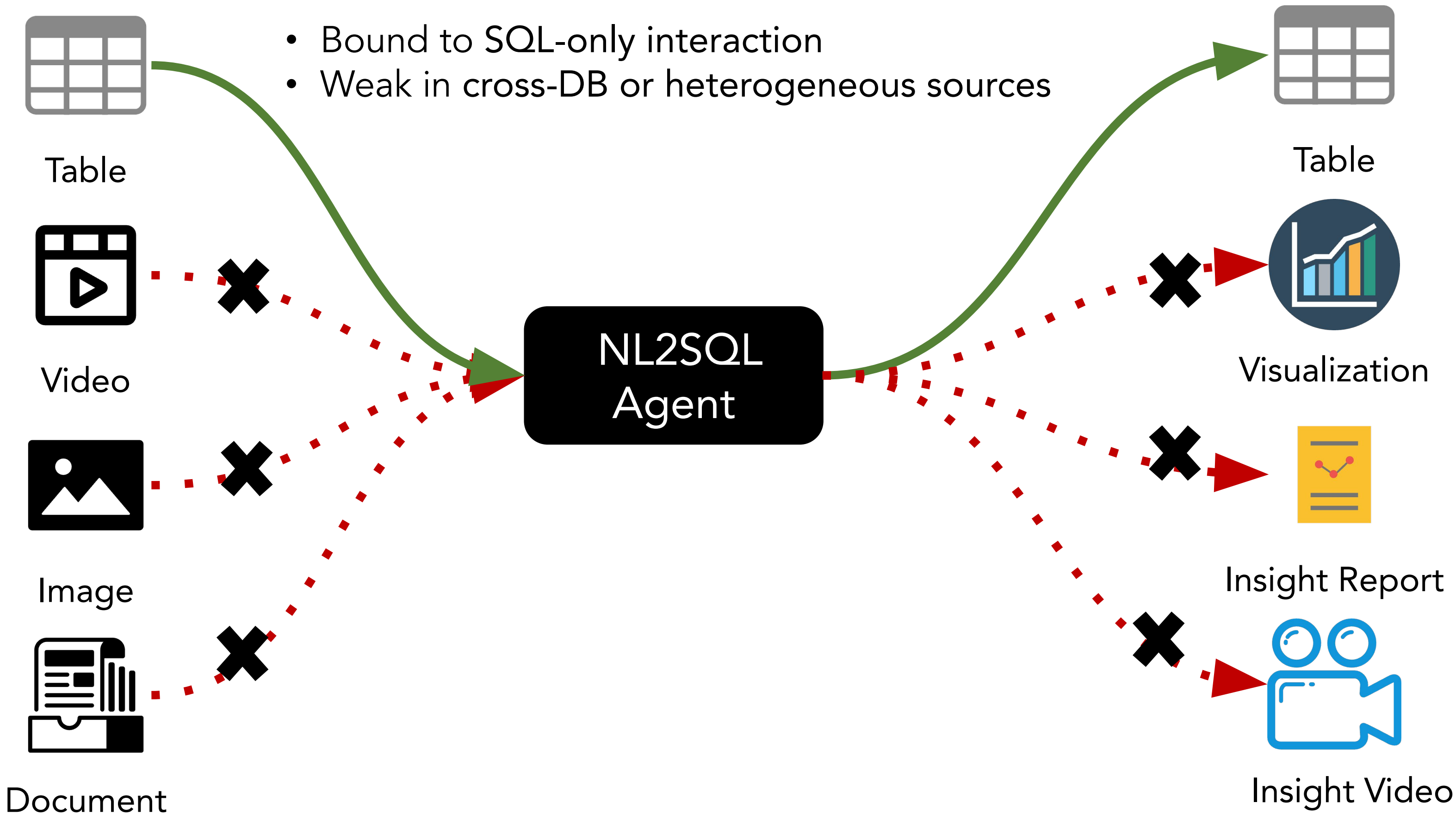
Liu. *et al.* (2026) “Business Logic-Driven Text-to-SQL Data Synthesis for Business Intelligence.” arXiv. Available at: https://arxiv.org/pdf/2601.14518.

Takeaways

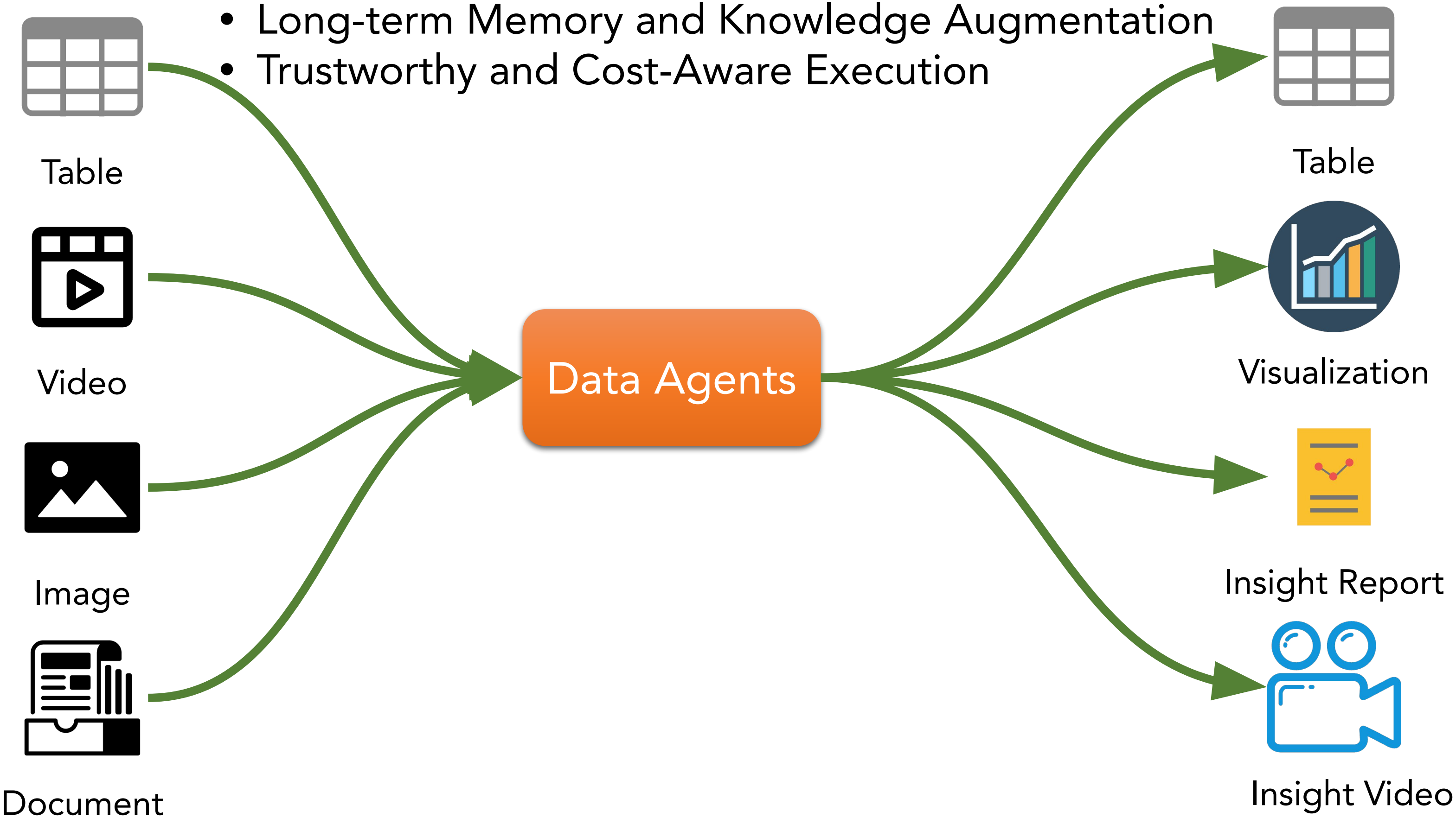
- The evaluation gap is not just “more realistic benchmarks,” but “realistic evaluation on the actual business database where the agent will be used.”

Are NL2SQL Agents Enough?

- Bound to SQL-only interaction
- Weak in cross-DB or heterogeneous sources



- Unified Query Interface: SQL -->Semantic Operators
- Multimodal Data Analysis
- Adaptive Reasoning and Orchestration
- Long-term Memory and Knowledge Augmentation
- Trustworthy and Cost-Aware Execution



Data Agent

- Data Agent: designed to autonomously carry out data-related tasks with capabilities for knowledge comprehension, automatic planning, and self-reflection of LLMs

- Challenges:
  - How can data agents understand queries, data, other agents, and tools?
  - How can data agents orchestrate effective and efficient pipelines to bridge the gaps between user requirements and underlying heterogeneous data?
  - How to schedule and coordinate agents/tools to improve effectiveness?

Guoliang Li et al. Data+AI: LLM4Data and Data4LLM. SIGMOD 2025
https://dbgroup.cs.tsinghua.edu.cn/ligl/papers/SIGMOD2025-Data+LLM.pdf